

Introduction to Spring AI

AD

Last updated: February 13, 2026



Written by: Hardik Singh Behl (<https://www.baeldung.com/author/hardiksinghbeh1>)



Reviewed by: Michal Aibin (<https://www.baeldung.com/editor/michal-author>)

Artificial Intelligence (<https://www.baeldung.com/category/artificial-intelligence>)

Spring AI (<https://www.baeldung.com/category/spring/spring-ai>)

LLM (<https://www.baeldung.com/tag/llm>) **reference** ›

1. Overview

Modern applications are increasingly using Large Language Models (LLMs) (</cs/large-language-models>) to build solutions that go beyond traditional programming capabilities. However, integrating these models into our applications often involves dealing with complex APIs, managing different AI providers, and handling various configuration challenges.

Spring AI (<https://github.com/spring-projects/spring-ai>), a new addition to the Spring ecosystem, addresses these issues by providing a common abstraction layer for working with different AI providers using the familiar Spring programming patterns.

It eliminates the need to explicitly use provider-specific SDKs and enables us to switch between different models without changing our application code.

In this tutorial, **we'll practically explore the fundamental concepts of Spring AI by building a basic poem generation service.**

2. Setting up the Project

For our demonstration, **we'll be building our poem generation service using OpenAI's GPT-5** (<https://openai.com/index/introducing-gpt-5/>) model.

However, Spring AI supports models from various other providers like Anthropic ([/spring-ai-anthropic-claude-models/](https://spring-ai-anthropic-claude-models/)), DeepSeek ([/spring-ai-deepseek-cot/](https://spring-ai-deepseek-cot/)), and even local LLMs via Hugging Face or Ollama ([/spring-ai-ollama-hugging-face-models/](https://spring-ai-ollama-hugging-face-models/)). We can choose the model that best suits our requirements as the specific AI model is irrelevant for this implementation.

2.1. Dependencies

Let's start by adding the necessary dependency to our project's *pom.xml* file:

```
<dependency>
  <groupId>org.springframework.ai</groupId>
  <artifactId>spring-ai-starter-model-openai</artifactId>
  <version>1.0.1</version>
</dependency>
```

The OpenAI starter dependency (<https://mvnrepository.com/artifact/org.springframework.ai/spring-ai-starter-model-openai>) is a wrapper around OpenAI's Chat Completions API (<https://platform.openai.com/docs/api-reference/chat>), and we'll use it to interact with the GPT-5 model in our application.

2.2. Configuring LLM Properties

Next, let's configure our OpenAI API key (<https://platform.openai.com/api-keys>) and chat model in the *application.yaml* file:

```
spring:
  ai:
    openai:
      api-key: ${OPENAI_API_KEY}
    chat:
      options:
        model: gpt-5
        temperature: 1
```

We use the *\$\$* property placeholder to load the value of our API Key from an environment variable ([/spring-boot-properties-environment#use-environment-variable-in-applicationyaml-file](https://spring-boot-properties-environment#use-environment-variable-in-applicationyaml-file)).



Next, we specify *gpt-5* as the model ID. We can use a different model (<https://platform.openai.com/docs/models>) based on requirements.

Additionally, we set the *temperature* ([/cs/gpt-model-temperature](https://community.openai.com/t/temperature-in-gpt-5-models/1337133/6)) to *1* since the model we've configured only accepts this default value (<https://community.openai.com/t/temperature-in-gpt-5-models/1337133/6>).

3. Building a Poem Generation Service

With our configurations in place, let's build a service that generates poems using the configured LLM. We'll start with a basic implementation and gradually refactor it to use more advanced Spring AI features.

3.1. Using *ChatClient* to Communicate With LLM

In Spring AI, the *ChatClient* class serves as the main entry point for interacting with any model we configure.

We can obtain an instance of it using the *ChatClient.Builder* bean, which the framework automatically creates for us based on the properties we configure in our *application.yaml* file.



Let's use this to create a new *PoetryService* class:

```
private final ChatClient chatClient;

PoetryService(ChatClient.Builder chatClientBuilder) {
    this.chatClient = chatClientBuilder.build();
}

String generate() {
    return chatClient
        .prompt("Write a playful haiku about morning coffee following the traditional 5-7-5 syllable structure.")
        .call()
        .content();
}
```

Here, we inject the *ChatClient.Builder* into our service's constructor and use it to build a *ChatClient* instance.

Next, in our *generate()* method, we use the *prompt()* method of *chatClient* to send a prompt requesting a haiku (<https://www.britannica.com/art/haiku>).

Then, we invoke the *call()* method to execute the request against the configured LLM, and *content()* to extract the generated text as a simple *String*.

3.2. Refactoring Using *PromptTemplate* and Structured Output

While our initial implementation works, it's limited to generating haikus about coffee with a fixed prompt. Also, we return a plain string response that can be difficult to work with for clients.

To address these limitations, we'll refactor our service to use a prompt template where we can dynamically substitute genre and theme values at runtime and map the LLM's response to a structured Java object.

First, let's define a *Poem* record (/java-record-keyword) to represent the structure of our output:

```
record Poem {
    String title,
    String content,
    String genre,
    String theme) {}
```

We define the record with fields for *title*, *content*, *genre*, and *theme* to represent the structured response we expect from the LLM.

AD

Daar zijn ze dan: de leads waar je zo hard voor gewerkt hebt,...

Daar zijn ze dan: de leads waar je zo hard voor gewerkt hebt, rondklokkend op je website. En nu? Dankzij Leadinfo weet jij exact welke bedrijven interesse tonen in jouw product ...

LEARN MORE

SPONSORED BY
COMBELL



(https://ads.freestar.co

utm_campaign=branding&utm_medium=display&utm_source=bae

Next, let's refactor our service method:

```
private final static PromptTemplate PROMPT_TEMPLATE
    = new PromptTemplate("Write a {genre} haiku about {theme} following the traditional 5-7-5 syllable structure.");

Poem generate(String genre, String theme) {
    Prompt prompt = PROMPT_TEMPLATE
        .create(Map.of(
            "genre", genre,
            "theme", theme));
    return chatClient
        .prompt(prompt)
        .call()
        .entity(Poem.class);
}
```

s://ads.freestar.com/?utm_campaign=branding&utm_medium=display&utm_source=baeldung.com&utm_content=baeldung_inconte

In our refactored version, we replace the hardcoded prompt with a *PromptTemplate* that contains placeholders for *genre* and *theme*. In the *generate()* method, we now expect these values in the method parameters and use them to create a *Prompt* instance.

Additionally, we replace the *content()* method with *entity()* where we specify our *Poem* record. **Spring AI will automatically add instructions to the prompt to direct the LLM into generating a response that can be mapped to this record.**

3.3. Exposing a REST API and Handling Errors

Now that we've implemented our service layer, **let's expose a REST API on top of it:**

```
@PostMapping("/poems")
ResponseEntity<Poem> generate(@RequestBody PoemGenerationRequest request) {
    Poem response = poetryService.generate(request.genre, request.theme);
    return ResponseEntity.ok(response);
}

record PoemGenerationRequest(String genre, String theme) {}
```

(/)

s://ads.freestar.com/?utm_campaign=branding&utm_medium=display&utm_source=baeldung.com&utm_content=baeldung_inconte

Here, we simply define a POST `/poems` endpoint that accepts a *PoemGenerationRequest* record as the request body and simply delegates to our service layer to return the generated poem.

Additionally, as with communication with any external service, the configured LLM can sometimes fail. To handle such scenarios gracefully, **Spring AI provides an *OpenAiApiClientErrorException* that provides an abstraction over all OpenAI errors (<https://platform.openai.com/docs/guides/error-codes/api-errors>).**

Let's define an exception handler (`/exception-handling-for-rest-with-spring#2-global-exception-handling`) for this class:

```
private static final String LLM_COMMUNICATION_ERROR =
    "Unable to communicate with the configured LLM. Please try again later.";

@ExceptionHandler(OpenAiApiClientErrorException.class)
ProblemDetail handle(OpenAiApiClientErrorException exception) {
    logger.error("OpenAI returned an error.", exception);
    return ProblemDetail.forStatusAndDetail(HttpStatus.SERVICE_UNAVAILABLE, LLM_COMMUNICATION_ERROR);
}
```

Here, we intentionally avoid exposing the actual error details in the response to prevent leaking sensitive information about our infrastructure or API keys. Instead, **we log the full exception for debugging purposes and return a user-friendly message through the standardized *ProblemDetail* (`/spring-boot-return-errors-problemdetail`) response format.**

4. Testing Our Application

Finally, let's use the API endpoint we've exposed to interact with and test our application.

We'll use the HTTPie (`/httpie-http-client-command-line`) CLI to invoke the API:

```
http POST :8080/poems genre="frustrated" theme="code review comments"
```

Here, we send a POST request to our `/poems` endpoint with our desired *genre* and *theme*.

Let's see what we receive as a response:

```
{
  "title": "Nitpick Nightmare",
  "content": "Tabs versus spaces\nThey argue while prod is down\nPriorities... where?",
  "genre": "frustrated",
  "theme": "code review comments"
}
```

(/)

s://ads.freestar.com/?utm_campaign=branding&utm_medium=display&utm_source=baeldung.com&utm_content=baeldung_inconte

As we can see, we obtain a haiku that effectively captures the genre and theme we provided.

This confirms that our application correctly populates the prompt template and receives the LLM's output in a format that can be mapped to our *Poem* record.

5. Conclusion

In this article, we've explored integrating AI capabilities into a Spring Boot application using Spring AI.

We walked through the necessary configuration and implemented a poem generation service using OpenAI's GPT-5 model. We evolved our simple implementation from a string-based prompt to a more sophisticated solution using prompt templates and structured outputs.

While this introductory tutorial covers the fundamentals, **Spring AI offers extensive AI capabilities that can be explored in our collection of Spring AI tutorials (/category/spring/spring-ai).**

The code backing this article is available on GitHub. Once you're **logged in as a Baeldung Pro Member (/members/)**, start learning and coding on the project.

3 COMMENTS



Oldest ▾

View Comments

AD

Krijg je energie van richting geven, mensen meenemen...

Krijg je energie van richting geven, mensen meenemen en samen bouwen aan een sterke publieksbeleving? Als coördinator communicatie & publiekswerking combineer je...

LEARN MORE

SPONSORED BY CC SINT-NIKLAAS



(https://ads.freestar.com/?utm_campaign=branding&utm_medium=display&utm_source=baeldung.cc

COURSES

ALL COURSES (/COURSES/ALL-COURSES)

[BAELDUNG ALL ACCESS \(/COURSES/ALL-ACCESS\)](/COURSES/ALL-ACCESS)
[BAELDUNG ALL TEAM ACCESS \(/COURSES/ALL-ACCESS-TEAM\)](/COURSES/ALL-ACCESS-TEAM)
[LOGIN COURSE PLATFORM \(HTTPS://WWW.BAELDUNG.COM/MEMBERS/ACCOUNT\)](https://www.baeldung.com/members/account)

SERIES

[JAVA "BACK TO BASICS" TUTORIAL \(/JAVA-TUTORIAL\)](/JAVA-TUTORIAL)
[LEARN SPRING BOOT SERIES \(/SPRING-BOOT\)](/SPRING-BOOT)
[SPRING TUTORIAL \(/SPRING-TUTORIAL\)](/SPRING-TUTORIAL)
[GET STARTED WITH JAVA \(/GET-STARTED-WITH-JAVA-SERIES\)](/GET-STARTED-WITH-JAVA-SERIES)
[SPRING FRAMEWORK INTRODUCTION \(/SPRING-INTRO\)](/SPRING-INTRO)
[ALL ABOUT STRING IN JAVA \(/JAVA-STRING\)](/JAVA-STRING)
[SECURITY WITH SPRING \(/SECURITY-SPRING\)](/SECURITY-SPRING)

ABOUT

[ABOUT BAELDUNG \(/ABOUT\)](/ABOUT)
[THE FULL ARCHIVE \(/FULL-ARCHIVE\)](/FULL-ARCHIVE)
[EDITORS \(/EDITORS\)](/EDITORS)
[OUR PARTNERS \(/PARTNERS/\)](/PARTNERS/)
[PARTNER WITH BAELDUNG \(/PARTNERS/WORK-WITH-US\)](/PARTNERS/WORK-WITH-US)
[EBOOKS \(/LIBRARY/\)](/LIBRARY/)
[FAQ \(/LIBRARY/FAQ\)](/LIBRARY/FAQ)
 [BAELDUNG PRO \(/MEMBERS/\)](/MEMBERS/)

[TERMS OF SERVICE \(/TERMS-OF-SERVICE\)](/TERMS-OF-SERVICE)
[PRIVACY POLICY \(/PRIVACY-POLICY\)](/PRIVACY-POLICY)
[COMPANY INFO \(/BAELDUNG-COMPANY-INFO\)](/BAELDUNG-COMPANY-INFO)
[CONTACT \(/CONTACT\)](/CONTACT)

[PRIVACY PREFERENCES](#)